Name : Vivek Singh
RegNumber : 230905408
Roll Number: A50

Lab 4: Python

Q1) Write a python program to reverse a content a file and store it in another file.

```
file = ""
with open("./lab4/idk.txt", "r") as f:
file = f.read()
reverse = file[::-1]
with open("./lab4/oq1.txt", "w") as f:
f.write(reverse)
```

```
idk.txt:

cdfhg uvfgherkjgb iuver

vurg edfufvgr hvoirfg  hello world this is a line to test the working of this file
```

```
oq1.txt

 elif siht fo gnikrow eht tset ot enil a si siht dlrow olleh  gfriovh rgvfufde gruv

 revui bgjkrehgfvu ghfdc
```

Q2) Write a python program to implement binary search with recursion.

```
import sys
num = int(sys.argv[1])
a = [1,2,3,4,5,6,7,8,9]
def search(l, r, e):
m=(l+r)//2
if l==r:
return 1
if a[m]==e:
return 0
if a[m]<e:
return search(m+1, r, e)
if a[m]>e:
return search(l, r-1, e)
print(("Found", "Not Found")[search(0,8,num)])
```

output:
VI_WPL_A2@csl-11:~/Documents/230905408$ python3 lab4/q2.py 4
Found

Q3) Write a python program to sort words in alphabetical order.

```
def sort_words(words):
return sorted(words)

words_list = ["banana", "apple", "orange"]
sorted_words = sort_words(words_list)
print(sorted_words)
```

output: ['apple', 'banana', 'orange']

Q4) Write a Python class to get all possible unique subsets from a set of distinct integers Input:[4,5,6]
Output : [[], [6], [5], [5, 6], [4], [4, 6], [4, 5], [4, 5, 6]]

```
class SubsetGenerator:
def __init__(self, nums):
self.nums = nums

def subsets(self):
result = []
self._generate_subsets(0, [], result)
return result

def _generate_subsets(self, index, current, result):
result.append(current)
for i in range(index, len(self.nums)):
self._generate_subsets(i + 1, current + [self.nums[i]], result)

generator = SubsetGenerator([4, 5, 6])
print(generator.subsets())
```

o/p: [[], [4], [4, 5], [4, 5, 6], [4, 6], [5], [5, 6], [6]]

5. Write a Python class to find a pair of elements (indices of the two numbers) from a given array whose sum equals a specific target number.
Input: numbers= [10,20,10,40,50,60,70], target=50
Output: 3, 4.

```
class PairFinder:
def __init__(self, numbers):
self.numbers = numbers
def find_pair(self, target):
num_dict = {}
for index, num in enumerate(self.numbers):
complement = target - num
if complement in num_dict:
return num_dict[complement], index
```

```
num_dict[num] = index
return None
finder = PairFinder([10, 20, 10, 40, 50, 60, 70])
print(finder.find_pair(50))
```

output: (2, 3)

Q6. Write a Python class to implement pow(x, n).

```
class PowerCalculator:
def pow(self, x, n):
if n < 0:
return 1 / self._pow_helper(x, -n)
return self._pow_helper(x, n)

def _pow_helper(self, x, n):
if n == 0:
return 1
half = self._pow_helper(x, n // 2)
if n % 2 == 0:
return half * half
return half * half * x

calculator = PowerCalculator()
print(calculator.pow(2, 10))
```

output: 1024

Q7. Write a Python class which has two methods get_String and print_String. The get_String accept a string from the user and print_String print the string in upper case.

```
class StringManipulator:
def get_string(self):
self.user_string = input("Enter a string: ")

def print_string(self):
print(self.user_string.upper())

manipulator = StringManipulator()
manipulator.get_string()
manipulator.print_string()
```

output:
Enter a string: hello
HELLO